Laporan Tugas Besar 1

IF3170 - Inteligensi Artifisial



Dosen Pengampu: Dr. Edward Ferdian S.T, B.Eng, M.Sc.

Disusun oleh:	G48	NaturalStupidity
	13524024	Billie Bhaskara Wibawa
	13524048	Josh Reinhart Zidik
	13524088	Syaqina Octavia Rizha

DESKRIPSI PERSOALAN

Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal disini berarti memaksimalkan fungsi objektif.

Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Package	id	"RusdiPackage"	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar (seluruh bagian paket di luar)
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.
- Inisialisasi (*initial state*) random.
- Langkah neighbor (*move*) yang diperbolehkan tiap iterasi:

1. Menukar dua paket, baik di dalam maupun di luar truk
2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
3. Merotasi paket 90° terhadap sumbu *x*, *y*, atau *z*

- Representasi solusi (*final state*) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:

    1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar
    2. Bagaimana keadaan peti kemas pada truk
    3. *Value* pada *state* tersebut

Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- Objective function
  Permasalahan ini bertujuan untuk memaksimalkan *value* peti kemas pada truk.
- Kardus tidak bisa melayang dan menembus dinding peti kemas
  Melayang disini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i, y_i, z_i)$$

  dan dimensi

  $$(w_i, d_i, h_i)$$

  sehingga alas paket i adalah

  $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i,\ y_i \leq y < y_i + d_i\})$$

  Sebuah titik di alas memiliki *support* apabila:

  1. $Z_{i=0}$, artinya di alas truk, atau
  2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

  Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.

- Barang pecah-belah tidak boleh di bawah paket lain

  Barang pecah-belah tidak boleh menjadi *support*

- Truk tidak boleh mengangkut melebihi kapasitasnya

PEMBAHASAN

Representasi Persoalan

Representasi State

Setiap paket dalam sebuah *state* direpresentasikan melalui sebuah kelas bernama Package, sementara sebuah *state* direpresentasikan melalui kelas bernama TruckState. Kedua kelas tersebut dapat ditemukan di dalam *file source code* bernama `utils.py`. Setiap kelas memiliki atribut sesuai dengan spesifikasi persoalan kecuali untuk atribut "orientation" pada kelas Package, orientasi paket dilakukan secara langsung melalui modifikasi pada atribut width, length, dan height.

```python
class Package:
    def __init__(self, ident, pos, dim, value, weight, isFragile, eta, isInside):
        self.id = ident
        self.pos = pos.copy()
        self.dim = dim.copy()
        self.value = value
        self.weight = weight
        self.isFragile = isFragile
        self.eta = eta
        self.isInside = isInside
```

Representasi posisi dan dimensi dari kelas Package dan TruckState dilakukan menggunakan kelas *intermediate* yang mencakup atribut (x y z untuk posisi dan width length height untuk dimensi) dan metode (`move()` untuk posisi dan `rotate()` untuk dimensi) yang diperlukan. Selain itu, terdapat juga atribut tambahan isInside yang digunakan untuk menandakan jika paket pada TruckState berada di luar atau di dalam truk.

```python
class TruckState:
    def __init__(self, dim, cap, packs):
        self.dim = dim.copy()
        self.cap = cap
        self.packs = [
                        Package(
                            pack.id,
                            pack.pos,
                            pack.dim,
                            pack.value,
                            pack.weight,
                            pack.isFragile,
                            pack.eta,
                            pack.isInside
                        )
                        for pack in packs
                    ]
```

Pembangkitan State Tetangga

Spesifikasi persoalan mendefinisikan *state* tetangga merupakan *state* yang mengalami perubahan tepat satu kali dari *state* sendiri dengan jenis sebagai berikut:

a. Menukar dua paket, baik di dalam maupun di luar truk
b. Memindahkan sebuah paket ke suatu koordinat yang berbeda
c. Merotasi paket 90° terhadap sumbu x, y, atau z

Setiap poin perubahan direpresentasikan melalui metode di dalam kelas TruckState, yaitu move_pack(), rotate_pack(), dan swap_pack().

```
176  def move_pack(self, packid, newpos):
177      pack = self._find(packid)
178      if pack.isInside and pack.pos.equal(newpos.x, newpos.y, newpos.z):
179          return None
180      old = pack.copy()
181      pack.pos.move(newpos.x, newpos.y, newpos.z)
182      pack.isInside = True
183      try:
184          if self.validate_change([pack], [old]):
185              return self.copy()
186          else:
187              return None
188      finally:
189          pack.pos = old.pos
190          pack.isInside = old.isInside
```

```
191  def swap_pack(self, id1, id2):
192      if (id1 == id2):
193          return None
194      pack1, pack2 = self._find(id1), self._find(id2)
195      if not pack1.isInside and not pack2.isInside:
196          return None
197      olds = [pack1.copy(), pack2.copy()]
198      pack1.isInside, pack2.isInside = pack2.isInside, pack1.isInside
199      pack1.pos,pack2.pos = pack2.pos.copy(), pack1.pos.copy()
200      try:
201          if self.validate_change([pack1, pack2], olds):
202              return self.copy()
203          else:
204              return None
205      finally:
206          pack1.pos, pack2.pos = olds[0].pos, olds[1].pos
207          pack1.isInside, pack2.isInside = olds[0].isInside, olds[1].isInside
```

Setiap metode di atas akan mengembalikan TruckState yang baru dan memastikan bahwa *state* yang dikembalikan bersifat **valid**, *state* hasil perubahan tidak valid tidak akan dikembalikan (return None).

Pembangkitan *state* tetangga terbagi menjadi dua tergantung *output* yang diharapkan, seluruh *state* tetangga yang valid atau satu *state* acak yang valid. Pembangkitan seluruh *state* diimplementasikan melalui metode get_neighbors() sementara pembangkitan satu *state* acak diimplementasikan melalui metode get_random_neighbor(max_attempt: int) dengan parameter max_attempt untuk menandai jumlah percobaan dalam pembangkitan *state* acak.

```python
def get_neighbors(self):
    neighbors = []
    for pack in self.packs:

        for pos in self._move_candidates(pack):
            state = self.move_pack(pack.id, pos)
            if state is not None:
                neighbors.append(state)

        if pack.isInside:
            for axis in range(3):
                state = self.rotate_pack(pack.id, axis)
                if state is not None:
                    neighbors.append(state)

    for i in range(len(self.packs)):
        for j in range(i+1, len(self.packs)):
            state = self.swap_pack(self.packs[i].id, self.packs[j].id)
            if state is not None:
                neighbors.append(state)

    return neighbors
```

```python
def get_random_neighbor(self,max_attempts = 25):
    if not self.packs:
        return None

    for _ in range(max_attempts):
        selected_function = random.randint(1, 3)
        pack = random.choice(self.packs)

        match selected_function:
            case 1:  # move_pack
                    w, h, l = pack.dim.width, pack.dim.height, pack.dim.length
                    if (w <= self.dim.width
                        and h <= self.dim.height
                        and l <= self.dim.length):
                        for _ in range(10):
                            x = random.randint(0, self.dim.width - w)
                            z = random.randint(0, self.dim.length - l)
                            y = self.drop_height(pack, x, z)
                            if y + h <= self.dim.height:
                                state = self.move_pack(pack.id, Position(x, y, z))
                                if state is not None:
                                    return state
            case 2:  # rotate_pack
                    axes = [0, 1, 2]
                    random.shuffle(axes)
                    for axis in axes:
                        state = self.rotate_pack(pack.id, axis)
                        if state is not None:
                            return state
            case 3:  # swap_pack
                    others = [p.id for p in self.packs if p.id != pack.id]
                    random.shuffle(others)
                    for other_id in others:
                        state = self.swap_pack(pack.id, other_id)
                        if state is not None:
                            return state
    return None
```

Steepest Ascent Hill Climbing

Steepest Ascent Hill Climbing merupakan salah satu varian dari algoritma *hill climbing*, dilakukan dengan mengevaluasi *objective function* untuk setiap *neighbor* dari *state* yang sedang diperiksa saat ini kemudian mengambil *neighbor* dengan nilai *objective function* tertinggi.

Implementasi

Implementasi untuk *Steepest Ascent Hill Climbing* dapat ditemukan di dalam *source code* dengan nama hill_climbing.py pada fungsi bernama steepest_ascent(). Fungsi hanya memiliki parameter truck sebagai *state* awal dari proses pencarian. Pemilihan acak *state* awal sendiri dilakukan sebelum fungsi dipanggil, sehingga dipastikan bahwa truck sudah memiliki pemosisian paket yang acak. Algoritma akan berhenti saat mencapai sebuah *state* yang tidak memiliki *neighbor state* yang lebih baik, kemudian fungsi mengembalikan metrik tambahan "iterations" untuk jumlah iterasi sesuai dengan spesifikasi persoalan.

```python
def steepest_ascent(truck: TruckState) -> SearchResult:
    start_time = time.perf_counter()

    initial_state = truck.copy()
    current_state = truck.copy()

    maxval = current_state.count_value()
    prevval = maxval

    obj_history = [maxval]
    while True:
        states = current_state.get_neighbors()
        for state in states:
            curval = state.count_value()
            if curval > maxval:
                maxval = curval
                current_state = state
        if maxval == prevval:
            break
        prevval = maxval
        obj_history.append(current_state.count_value())

    duration = time.perf_counter() - start_time

    return SearchResult(
        initial_state=initial_state,
        final_state=current_state,
        obj_history=obj_history,
        duration=duration,
        metrics={
            "iterations": len(obj_history) - 1,
        }
    )
```

Stochastic Hill Climbing

Stochastic Ascent Hill Climbing merupakan salah satu varian dari algoritma *hill climbing*, dilakukan dengan memilih *neighbor state* secara acak kemudian memilih

neighbor state pertama yang memiliki nilai *objective function* yang lebih tinggi dari *state* saat ini. Proses pemilihan *neighbor state* acak tersebut akan berlangsung sejumlah parameter **n** yang ditentukan, algoritma akan dihentikan jika setelah **n** jumlah iterasi dilakukan tidak ditemukan *neighbor state* dengan nilai *objective function* lebih tinggi.

```python
def stochastic(truck: TruckState, max_iter: int = 50) -> SearchResult:
    start_time = time.perf_counter()

    initial_state = truck.copy()
    current_state = truck.copy()

    val = current_state.count_value()

    obj_history = [val]
    i = 0
    while i < max_iter:
        next_state = current_state.get_random_neighbor(200)
        if next_state is None:
            break
        if next_state.count_value() > current_state.count_value():
            current_state = next_state
            i = 0
        else:
            i += 1
        obj_history.append(current_state.count_value())

    duration = time.perf_counter() - start_time

    return SearchResult(
        initial_state=initial_state,
        final_state=current_state,
        obj_history=obj_history,
        duration=duration,
        metrics={
            "iterations": len(obj_history) - 1,
        }
    )
```

Implementasi

Implementasi untuk *Stochastic Hill Climbing* dapat ditemukan di dalam *source code* dengan nama hill_climbing.py pada fungsi bernama stochastic(). Fungsi memiliki parameter truck sebagai *state* awal dari proses pencarian dan parameter max_iter untuk menjadi konstanta jumlah maksimum iterasi yang dilakukan. Proses pengambilan *state* tetangga acak menggunakan fungsi get_random_neighbor() yang sudah didefinisikan sebelumnya. Algoritma akan berhenti saat percobaan pencarian *state* tetangga sudah mencapai parameter max_iter, kemudian fungsi mengembalikan metrik tambahan "iterations" untuk jumlah iterasi sesuai dengan spesifikasi persoalan.

Hill Climbing with Sideways Move

Hill Climbing with Sideways Move merupakan salah satu varian dari algoritma *hill climbing*, dilakukan dengan mengevaluasi *objective function* untuk setiap *neighbor* dari *state* yang sedang diperiksa saat ini kemudian mengambil *neighbor* dengan nilai *objective function* tertinggi. Perbedaan algoritma ini dengan *Steepest Ascent Hill Climbing* adalah ketika *state* merupakan *local optimum*, algoritma dapat memilih *state*

tetangga dengan nilai yang sama dengan nilai *state* saat ini. Terminasi algoritma ini dilakukan ketika jumlah gerakan *sideways* berurutan sudah mencapai parameter yang ditentukan.

Implementasi

Implementasi untuk *Hill Climbing with Sideways Move* dapat ditemukan di dalam *source code* dengan nama hill_climbing.py pada fungsi bernama sideways_move(). Fungsi memiliki parameter truck sebagai *state* awal dari proses pencarian dan parameter max_sideways untuk menjadi parameter jumlah maksimum gerakan *sideways* berturut-turut yang dapat dilakukan. Algoritma akan berhenti saat gerakan *sideways* berturut-turut sudah mencapai parameter max_sideways, kemudian fungsi mengembalikan metrik tambahan "iterations" untuk jumlah iterasi dan "sideways_count" untuk total jumlah gerakan *sideways*.

```python
def sideways_move(truck: TruckState, max_sideways = 50) -> SearchResult:
    start_time = time.perf_counter()

    initial_state = truck.copy()
    current_state = initial_state

    maxval = current_state.count_value()

    obj_history = [maxval]
    prevval = maxval
    side_count = 0
    total_side = 0
    while True:
        side_state = []
        for state in current_state.get_neighbors():
            curval = state.count_value()
            if curval > maxval:
                maxval = curval
                current_state = state
            elif curval == maxval:
                side_state.append(state)
        if maxval == prevval:
            if len(side_state) == 0 or side_count == max_sideways:
                break
            current_state = random.choice(side_state)
            side_count += 1
            total_side += 1
        else:
            side_count = 0
        prevval = maxval
        obj_history.append(current_state.count_value())

    duration = time.perf_counter() - start_time

    return SearchResult(
        initial_state=initial_state,
        final_state=current_state,
        obj_history=obj_history,
        duration=duration,
        metrics={
            "iterations": len(obj_history) - 1,
            "sideways_count": total_side
        }
    )
```

Random Restart Hill Climbing

Random Restart Hill Climbing merupakan salah satu varian dari algoritma *hill climbing* yang serupa dengan *Steepest Ascent Hill Climbing*. Algoritma ini berbeda pada jumlah iterasi *state* awal, ketika proses *hill climbing* sudah mencapai *local optimum*, algoritma ini akan membangkitkan *state* awal yang baru. Jumlah pembangkitan *state* awal

dibatasi menggunakan parameter **n**, algoritma akan diterminasi ketika jumlah pembangkitan sudah melewati parameter tersebut

```python
def random_restart(truck: TruckState, max_restart = 50) -> SearchResult:
    start_time = time.perf_counter()

    initial_state = truck.copy()
    current_state = truck.copy()
    best_state = initial_state

    restart_count = 0
    maxval = current_state.count_value()
    global_max = maxval
    obj_history = [maxval]

    prevval = maxval
    iter_per_restart = []
    iteration = 0
    while True:
        iteration += 1
        states = current_state.get_neighbors()
        for state in states:
            curval = state.count_value()
            if curval > maxval:
                maxval = curval
                current_state = state
        if maxval == prevval:
            iter_per_restart.append(iteration)
            iteration = 0
            if maxval > global_max:
                best_state = current_state
                global_max = maxval

            if restart_count == max_restart:
                break
            restart_count += 1
            current_state = randomize_pack(truck)
            maxval = current_state.count_value()
            prevval = current_state.count_value()

        prevval = maxval
        obj_history.append(current_state.count_value())

    duration = time.perf_counter() - start_time

    return SearchResult(
        initial_state=initial_state,
        final_state=best_state,
        obj_history=obj_history,
        duration=duration,
        metrics={
            "iterations": len(obj_history) - 1,
            "restarts": max_restart,
            "iter_per_restart": iter_per_restart
        }
    )
```

Implementasi

Implementasi untuk *Random Restart Hill Climbing* dapat ditemukan di dalam *source code* dengan nama hill_climbing.py pada fungsi bernama random_restart(). Fungsi memiliki parameter truck sebagai *state* awal dari proses pencarian dan parameter max_restart untuk menjadi parameter jumlah pembangkitan *state* awal baru yang dapat dilakukan. Algoritma tetap mencatat *state* yang terbaik meskipun terjadi pembangkitan *state* awal yang baru, algoritma akan berhenti ketika jumlah pembangkitan sudah mencapai max_restart kemudian mengembalikan metrik tambahan "iterations" untuk jumlah iterasi, "restarts" untuk jumlah pembangkitan *state* awal baru, dan "iter_per_restart" untuk jumlah iterasi untuk setiap *state* awal baru.

Simulated Annealing

Simulated Annealing merupakan algoritma optimasi yang mengevaluasi objective function dari neighbor suatu state dan menentukan apakah solusi tersebut diterima berdasarkan nilai objektif dan suhu (temperature) saat ini. Algoritma ini dapat menerima solusi yang lebih buruk dengan probabilitas tertentu untuk menghindari

terjebak pada local optimum, dengan probabilitas penerimaan yang semakin menurun seiring turunnya temperature.

Implementasi

Implementasi *Simulated Annealing* dapat ditemukan pada file simulated_annealing.py pada fungsi simulate_annealing(). Fungsi tersebut menerima T0 (temp awal) dan alpha yang bersama-sama mengontrol temperature T = T0 * alpha^t. Pada setiap iterasi, algoritma mengambil satu random neighbor via get_random_neighbor() (yang sudah efisien karena tidak generate semua tetangga), menghitung delta = fitness_tetangga - fitness_saat_ini, dan selalu menerima jika delta > 0; jika delta ≤ 0, diterima dengan probabilitas exp(delta/T).

Variabel cache menyimpan daftar tetangga saat get_random_neighbor gagal (hanya 1 tetangga valid) agar tidak memanggil ulang fungsi yang memiliki komputasi mahal. Loop berhenti saat T < 1e-3. Metrics mengembalikan stuck_count (jumlah penolakan), e_delta_t_history (probabilitas penerimaan tiap iterasi), dan final_current_value vs best_value untuk analisis perilaku exploration-exploitation.

```python
def simmulate_annealing(truck: TruckState, T0: float = 200.0, alpha: float = 0.995) -> SearchResult:
    start_time = time.perf_counter()

    initial_state = truck.copy()
    current_state = truck.copy()
    best_state = current_state.copy()
    cache = None

    """
    The cache is for the edgecase where there is only 1 valid neighbor
    from the current state (almost doomed state but not quite) and then
    that 1 valid neighbor is rejected (thus current state is the same).
    It's to prevent calling get_neighbors() twice on the same state (since
    that function is expensive af)
    """

    obj_history = [current_state.count_value()]
    e_delta_t_history = []
    stuck_count = 0

    t = 1
    while True:
        T = schedule(t, T0=T0, alpha=alpha)
        t += 1
        if T < 1e-3:
            break

        if cache is None:
            next_state = current_state.get_random_neighbor(max_attempts=100)
            if next_state is None:
                all_neighbors = current_state.get_neighbors()
                cache = all_neighbors
                if not all_neighbors:
                    break
                next_state = random.choice(all_neighbors)
        else:
            next_state = random.choice(cache)
```

```python
        delta = next_state.count_value() - current_state.count_value()
        if delta > 0:
            e_delta_t_history.append(1.0)
            current_state = next_state
            cache = None
            if next_state.count_value() > best_state.count_value():
                best_state = next_state.copy()
        else:
            prob = accept_probability(delta, T)
            e_delta_t_history.append(prob)
            if random.random() < prob:
                current_state = next_state
                cache = None
            else:
                stuck_count += 1

        obj_history.append(current_state.count_value())

    duration = time.perf_counter() - start_time

    return SearchResult(
        initial_state=initial_state,
        final_state=best_state,
        obj_history=obj_history,
        duration=duration,
        metrics={
            "iterations": len(obj_history) - 1,
            "e_delta_t_history": e_delta_t_history,
            "stuck_count": stuck_count,
            "final_current_value": current_state.count_value(),
            "best_value": best_state.count_value(),
        }
    )
```

Genetic Algorithm

Genetic Algorithm (GA) merupakan algoritma optimasi yang terinspirasi dari proses evolusi alami. Algoritma ini bekerja dengan membentuk sekumpulan solusi (population), mengevaluasi nilai fitness setiap solusi, lalu menghasilkan generasi baru melalui proses selection, crossover, dan mutation untuk memperoleh solusi yang semakin optimal.

Implementasi

- Selection

  selection(population) menerapkan roulette wheel. Setiap kromosom mendapat peluang dipilih proporsional ke count_value()-nya. Jika total fitness nol (semua kromosom invalid/zero), fallback ke uniform random agar algoritma tidak stuck.

```python
def selection(population):
    #Roulette Wheel Selection (bobot berdasarkan fitness)
    if not population:
        return []
    fitness_values = [ch.count_value() for ch in population]
    total_fitness = sum(fitness_values)

    if total_fitness == 0:
            return random.choices(population, k=len(population))

    selection_probs = [f / total_fitness for f in fitness_values]
    selected_indices = random.choices(range(len(population)), weights=selection_probs, k=len(population))

    return [population[i] for i in selected_indices]
```

- Crossover

  crossover_one_child(p1, p2, base) membangun child dengan menggabungkan gen per paket (indeks i) dari dua parent: indeks ganjil ambil dari p1 dulu, genap dari p2 dulu (alternating agar adil). Setiap gen disalin, lalu validate_state() dicek, jika valid, lanjut; jika tidak, coba gen parent lain; jika keduanya invalid, paket dilepaskan (isInside=False). crossover() memanggil fungsi ini dua kali (balik parent) untuk menghasilkan dua anak bersaudara.

  Desain crossover ini dipilih karena menjaga validitas hard constraints (tidak overlap, tidak floating, fragile di bawah, kapasitas berat), crossover standar (1-point, 2-point) hampir pasti menghasilkan state invalid di ruang solusi 3D bin-packing ini.

```python
def crossover_one_child(parent1: TruckState, parent2: TruckState, base_state: TruckState) -> TruckState:
    child = base_state.copy()
    n = len(child.packs)

    for i in range(n):

        # i ganjil -> cek p1 dulu
        if i % 2 == 1:
            first, second = parent1, parent2
        else:
            first, second = parent2, parent1

        child.packs[i] = first.packs[i].copy()
        if child.validate_state():
            continue
        #jika parent 1 tidak bikin valid, coba parent 2
        child.packs[i] = second.packs[i].copy()
        if child.validate_state():
            continue

        child.packs[i].isInside = False

    return child

def crossover(parent1: TruckState, parent2: TruckState, base_state: TruckState) -> Tuple[TruckState, TruckState]:
    child1 = crossover_one_child(parent1, parent2, base_state)
    child2 = crossover_one_child(parent2, parent1, base_state)
    return child1, child2
```

- Mutation

  mutate(state, rate) dengan probabilitas rate meminta satu random neighbor melalu get_random_neighbor(max_attempts=20) (move/rotate/swap acak). Jika neighbor valid, terjadi mutasi; jika tidak atau probabilitas gagal, kembalikan state asli.

```python
def mutate(state: TruckState, mutation_rate: float = 0.20) -> TruckState:
    #apply 1 random neighbor move
    if random.random() > mutation_rate: #Lebih sering tidak mutate
        return state
    neighbor = state.get_random_neighbor(max_attempts=20)

    return neighbor if neighbor else state
```

- Algoritma utama

  Loop utama:

  1. evaluasi fitness seluruh populasi, catat max_fit, avg_fit, gen_best;
  2. elitism — hanya saat pop_size ganjil, simpan gen_best (best generasi ini) sebagai elite 1 individu, memastikan best fitness non-decreasing (solusi terbaik tidak hilang); saat genap tidak ada elitism (design choice agar pop_size - elitism genap -> pasangan crossover pas);

3. selection -> crossover -> mutate menghasilkan pop_size - elitism anak, dengan safety loop (max_attempts = pop_size*10) + fallback isi sisa dengan gen_best agar ukuran populasi konstan;
4. populasi baru = elites + offspring.

Terakhir return SearchResult dengan obj_history (best per generasi), avg_fitness, best_state_per_gen untuk visualisasi konvergensi. Elitism kondisional dipakai untuk menjaga konsistensi ukuran offspring (genap).

```python
def genetic_algorithm(base_truck: TruckState, pop_size: int, iterations: int, mutation_rate: float, seed: int = 21) -> SearchResult:
    start_time = time.perf_counter()

    empty_truck = base_truck.copy()
    for p in empty_truck.packs:
        p.isInside = False
        p.pos.move(0, 0, 0)

    population = generate_population(empty_truck, pop_size, seed=seed)
    initial_state = max(population, key=lambda s: s.count_value()).copy()

    history_max = []
    history_avg = []
    best_state_per_gen = []

    for _ in range(iterations):
        # Evaluate
        fitness = [s.count_value() if s.validate_state() else 0 for s in population]
        max_fit = max(fitness)
        avg_fit = sum(fitness) / len(fitness)
        gen_best = population[fitness.index(max_fit)]

        history_max.append(max_fit)
        history_avg.append(avg_fit)
        best_state_per_gen.append(gen_best.copy())

        # Elitism hanya jika pop_size ganjil
        if pop_size % 2 == 1:
            elitism = 1
            elites = [population[-1].copy()]
        else:
            elitism = 0
            elites = []

        # Selection
        selected_pop = selection(population)

        # Crossover
        next_gen = []

        idx = 0
        attempts = 0
        max_attempts = max(pop_size * 10, 100)
        n_parents = len(selected_pop)
        while len(next_gen) < pop_size - elitism and attempts < max_attempts:
            p1 = selected_pop[idx % n_parents]
            p2 = selected_pop[(idx + 1) % n_parents]
            idx += 2
            attempts += 1

            c1, c2 = crossover(p1, p2, base_truck)
            for child in (c1, c2):
                if child:
                    child = mutate(child, mutation_rate)
                    if child.validate_state():
                        next_gen.append(child)
                        if len(next_gen) >= pop_size - elitism:
                            break

        # gabung dengan elit nya
        while len(next_gen) < pop_size - elitism:
            next_gen.append(gen_best.copy())

        # Next generation
        population = elites + next_gen[:pop_size - elitism]
        assert len(population) == pop_size #pastiin jumlah sesuai

    duration = time.perf_counter() - start_time
    final_state = gen_best

    # Results metrics tambahan
    metrics = {
        'avg_fitness': history_avg,
        'avg_obj_history': history_avg,
        'best_fitness_per_gen': history_max,
        'best_state_per_gen': best_state_per_gen,
        'pop_size': pop_size,
        'iterations': iterations,
        'max_iter': iterations,
        'mutation_rate': mutation_rate,
        'final_fitness': final_state.count_value(),
    }

    return SearchResult(
        initial_state=initial_state,
        final_state=final_state,
        obj_history=history_max,
        duration=duration,
        metrics=metrics
    )
```

Eksperimen

	Test Case 1
Steepest Ascent Hill Climbing	"Run": "Run #1", "Objective Awal": 386, "Objective Akhir (Best)": 738, "Peningkatan (Delta)": 352, "Paket Masuk (Awal → Akhir)": "8 → 12", "Berat Akhir / Cap": "498 / 500", "Durasi (detik)": 1.7353, "Total Iterasi": 6
Stochastic Hill Climbing	"Run": "Run #1", "Objective Awal": 386, "Objective Akhir (Best)": 596, "Peningkatan (Delta)": 210, "Paket Masuk (Awal → Akhir)": "8 → 11", "Berat Akhir / Cap": "498 / 500", "Durasi (detik)": 0.0245, "Total Iterasi": 130
Hill Climbing with Sideways Move	"Run": "Run #1", "Objective Awal": 386, "Objective Akhir (Best)": 884, "Peningkatan (Delta)": 498, "Paket Masuk (Awal → Akhir)": "8 → 13", "Berat Akhir / Cap": "496 / 500", "Durasi (detik)": 12.698, "Total Iterasi": 103, "Jumlah Sideways Move": 94
Random Restart Hill Climbing	"Run": "Run #1", "Objective Awal": 386, "Objective Akhir (Best)": 811, "Peningkatan (Delta)": 425, "Paket Masuk (Awal → Akhir)": "8 → 12", "Berat Akhir / Cap": "486 / 500", "Durasi (detik)": 16.4137, "Total Iterasi": 70,

	"Banyak Restart": 10, "Iterasi per Restart": "[7, 7, 7, 5, 5, 7, 7, 6, 8, 5, 7]"
Simulated Annealing	"Run": "Run #1", "Objective Awal": 386, "Objective Akhir (Best)": 792, "Peningkatan (Delta)": 406, "Paket Masuk (Awal → Akhir)": "8 → 13", "Berat Akhir / Cap": "494 / 500", "Durasi (detik)": 0.6423, "Total Iterasi": 2435, "Frekuensi Stuck Local Optima": 246, "Nilai Akhir Current State": 792
Genetic Algorithm	

	Test Case 2
Steepest Ascent Hill Climbing	"Run": "Run #1", "Objective Awal": 339, "Objective Akhir (Best)": 426, "Peningkatan (Delta)": 87, "Paket Masuk (Awal → Akhir)": "7 → 7", "Berat Akhir / Cap": "196 / 200", "Durasi (detik)": 0.149, "Total Iterasi": 2
Stochastic Hill Climbing	"Run": "Run #1", "Objective Awal": 339, "Objective Akhir (Best)": 529, "Peningkatan (Delta)": 190, "Paket Masuk (Awal → Akhir)": "7 → 9", "Berat Akhir / Cap": "193 / 200", "Durasi (detik)": 0.01, "Total Iterasi": 112
Hill Climbing	"Run": "Run #1",

with Sideways Move	"Objective Awal": 339, "Objective Akhir (Best)": 549, "Peningkatan (Delta)": 210, "Paket Masuk (Awal → Akhir)": "7 → 9", "Berat Akhir / Cap": "185 / 200", "Durasi (detik)": 2.4343, "Total Iterasi": 88, "Jumlah Sideways Move": 78
Random Restart Hill Climbing	"Run": "Run #1", "Objective Awal": 339, "Objective Akhir (Best)": 549, "Peningkatan (Delta)": 210, "Paket Masuk (Awal → Akhir)": "7 → 9", "Berat Akhir / Cap": "185 / 200", "Durasi (detik)": 1.585, "Total Iterasi": 44, "Banyak Restart": 10, "Iterasi per Restart": "[3, 2, 4, 3, 4, 4, 5, 5, 3, 6, 6]"
Simulated Annealing	"Run": "Run #1", "Objective Awal": 339, "Objective Akhir (Best)": 549, "Peningkatan (Delta)": 210, "Paket Masuk (Awal → Akhir)": "7 → 9", "Berat Akhir / Cap": "185 / 200", "Durasi (detik)": 0.2883, "Total Iterasi": 2435, "Frekuensi Stuck Local Optima": 365, "Nilai Akhir Current State": 549
Genetic Algorithm	**Population control:** { "algorithm": "Genetic Algorithm", "total_runs": 3, "experiment_logs": [{ "Run": "Pop=30, Iter=50", "Objective Awal": 526, "Objective Akhir (Best)": 66,

```
      "Peningkatan (Delta)": -460,
      "Paket Masuk (Awal → Akhir)": "9 → 1",
      "Berat Akhir / Cap": "63 / 200",
      "Durasi (detik)": 0.2835,
      "Total Iterasi": 50,
      "Jumlah Populasi": 30,
      "Banyak Iterasi": 50,
      "Rata-rata Fitness Akhir": 50.1,
      "Mutation Rate": 0.2
    },
    {
      "Run": "Pop=30, Iter=100",
      "Objective Awal": 526,
      "Objective Akhir (Best)": 77,
      "Peningkatan (Delta)": -449,
      "Paket Masuk (Awal → Akhir)": "9 → 3",
      "Berat Akhir / Cap": "101 / 200",
      "Durasi (detik)": 0.6022,
      "Total Iterasi": 100,
      "Jumlah Populasi": 30,
      "Banyak Iterasi": 100,
      "Rata-rata Fitness Akhir": 53.3,
      "Mutation Rate": 0.2
    },
    {
      "Run": "Pop=30, Iter=200",
      "Objective Awal": 526,
      "Objective Akhir (Best)": 133,
      "Peningkatan (Delta)": -393,
      "Paket Masuk (Awal → Akhir)": "9 → 2",
      "Berat Akhir / Cap": "45 / 200",
      "Durasi (detik)": 1.1098,
      "Total Iterasi": 200,
      "Jumlah Populasi": 30,
      "Banyak Iterasi": 200,
      "Rata-rata Fitness Akhir": 55.2,
      "Mutation Rate": 0.2
    }
  ]
}
```

Iteration Control:


```
{
  "algorithm": "Genetic Algorithm",
  "total_runs": 3,
  "experiment_logs": [
    {
      "Run": "Pop=20, Iter=100",
      "Objective Awal": 526,
      "Objective Akhir (Best)": 125,
      "Peningkatan (Delta)": -401,
      "Paket Masuk (Awal → Akhir)": "9 → 3",
      "Berat Akhir / Cap": "99 / 200",
      "Durasi (detik)": 0.46,
      "Total Iterasi": 100,
      "Jumlah Populasi": 20,
      "Banyak Iterasi": 100,
      "Rata-rata Fitness Akhir": 68.85,
      "Mutation Rate": 0.2
    },
    {
      "Run": "Pop=50, Iter=100",
      "Objective Awal": 526,
      "Objective Akhir (Best)": 154,
      "Peningkatan (Delta)": -372,
      "Paket Masuk (Awal → Akhir)": "9 → 3",
      "Berat Akhir / Cap": "59 / 200",
      "Durasi (detik)": 0.965,
      "Total Iterasi": 100,
      "Jumlah Populasi": 50,
      "Banyak Iterasi": 100,
      "Rata-rata Fitness Akhir": 64.3,
      "Mutation Rate": 0.2
    },
    {
      "Run": "Pop=100, Iter=100",
      "Objective Awal": 526,
      "Objective Akhir (Best)": 435,
      "Peningkatan (Delta)": -91,
      "Paket Masuk (Awal → Akhir)": "9 → 6",
      "Berat Akhir / Cap": "152 / 200",
```

	"Durasi (detik)": 2.0252, "Total Iterasi": 100, "Jumlah Populasi": 100, "Banyak Iterasi": 100, "Rata-rata Fitness Akhir": 320.51, "Mutation Rate": 0.2 }] }

	Test Case 3
Steepest Ascent Hill Climbing	"Run": "Run #1", "Objective Awal": 408, "Objective Akhir (Best)": 587, "Peningkatan (Delta)": 179, "Paket Masuk (Awal → Akhir)": "7 → 8", "Berat Akhir / Cap": "194 / 200", "Durasi (detik)": 0.4025, "Total Iterasi": 5
Stochastic Hill Climbing	"Run": "Run #1", "Objective Awal": 408, "Objective Akhir (Best)": 587, "Peningkatan (Delta)": 179, "Paket Masuk (Awal → Akhir)": "7 → 8", "Berat Akhir / Cap": "194 / 200", "Durasi (detik)": 0.0138, "Total Iterasi": 228
Hill Climbing with Sideways Move	"Run": "Run #1", "Objective Awal": 408, "Objective Akhir (Best)": 587, "Peningkatan (Delta)": 179, "Paket Masuk (Awal → Akhir)": "7 → 8", "Berat Akhir / Cap": "194 / 200", "Durasi (detik)": 3.2688, "Total Iterasi": 55, "Jumlah Sideways Move": 50

Random Restart Hill Climbing	"Run": "Run #1", "Objective Awal": 408, "Objective Akhir (Best)": 587, "Peningkatan (Delta)": 179, "Paket Masuk (Awal → Akhir)": "7 → 8", "Berat Akhir / Cap": "194 / 200", "Durasi (detik)": 3.0641, "Total Iterasi": 40, "Banyak Restart": 10, "Iterasi per Restart": "[6, 5, 5, 5, 4, 2, 5, 2, 3, 3, 1]"
Simulated Annealing	"Run": "Run #1", "Objective Awal": 408, "Objective Akhir (Best)": 544, "Peningkatan (Delta)": 136, "Paket Masuk (Awal → Akhir)": "7 → 9", "Berat Akhir / Cap": "199 / 200", "Durasi (detik)": 0.2444, "Total Iterasi": 2435, "Frekuensi Stuck Local Optima": 137, "Nilai Akhir Current State": 544
Genetic Algorithm	Population control: { "algorithm": "Genetic Algorithm", "total_runs": 9, "experiment_logs": [{ "Run": "Pop=20, Iter=50 (Run #1)", "Objective Awal": 556, "Objective Akhir (Best)": 324, "Peningkatan (Delta)": -232, "Paket Masuk (Awal → Akhir)": "9 → 5", "Berat Akhir / Cap": "173 / 200", "Durasi (detik)": 0.4698, "Total Iterasi": 50, "Jumlah Populasi": 20, "Banyak Iterasi": 50, "Rata-rata Fitness Akhir": 265.6,

```
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=20, Iter=50 (Run #2)",
    "Objective Awal": 473,
    "Objective Akhir (Best)": 411,
    "Peningkatan (Delta)": -62,
    "Paket Masuk (Awal → Akhir)": "7 → 6",
    "Berat Akhir / Cap": "164 / 200",
    "Durasi (detik)": 0.5697,
    "Total Iterasi": 50,
    "Jumlah Populasi": 20,
    "Banyak Iterasi": 50,
    "Rata-rata Fitness Akhir": 322.15,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=20, Iter=50 (Run #3)",
    "Objective Awal": 495,
    "Objective Akhir (Best)": 449,
    "Peningkatan (Delta)": -46,
    "Paket Masuk (Awal → Akhir)": "7 → 7",
    "Berat Akhir / Cap": "193 / 200",
    "Durasi (detik)": 0.6838,
    "Total Iterasi": 50,
    "Jumlah Populasi": 20,
    "Banyak Iterasi": 50,
    "Rata-rata Fitness Akhir": 305.7,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=20, Iter=100 (Run #1)",
    "Objective Awal": 556,
    "Objective Akhir (Best)": 412,
    "Peningkatan (Delta)": -144,
    "Paket Masuk (Awal → Akhir)": "9 → 6",
    "Berat Akhir / Cap": "183 / 200",
    "Durasi (detik)": 1.0596,
    "Total Iterasi": 100,
    "Jumlah Populasi": 20,
```

```
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 394.4,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=20, Iter=100 (Run #2)",
    "Objective Awal": 473,
    "Objective Akhir (Best)": 424,
    "Peningkatan (Delta)": -49,
    "Paket Masuk (Awal → Akhir)": "7 → 6",
    "Berat Akhir / Cap": "142 / 200",
    "Durasi (detik)": 1.0913,
    "Total Iterasi": 100,
    "Jumlah Populasi": 20,
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 354.9,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=20, Iter=100 (Run #3)",
    "Objective Awal": 495,
    "Objective Akhir (Best)": 410,
    "Peningkatan (Delta)": -85,
    "Paket Masuk (Awal → Akhir)": "7 → 7",
    "Berat Akhir / Cap": "192 / 200",
    "Durasi (detik)": 1.1974,
    "Total Iterasi": 100,
    "Jumlah Populasi": 20,
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 370.45,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=20, Iter=200 (Run #1)",
    "Objective Awal": 556,
    "Objective Akhir (Best)": 408,
    "Peningkatan (Delta)": -148,
    "Paket Masuk (Awal → Akhir)": "9 → 6",
    "Berat Akhir / Cap": "159 / 200",
    "Durasi (detik)": 2.0537,
```

```
      "Total Iterasi": 200,
      "Jumlah Populasi": 20,
      "Banyak Iterasi": 200,
      "Rata-rata Fitness Akhir": 345.2,
      "Mutation Rate": 0.2
    },
    {
      "Run": "Pop=20, Iter=200 (Run #2)",
      "Objective Awal": 473,
      "Objective Akhir (Best)": 465,
      "Peningkatan (Delta)": -8,
      "Paket Masuk (Awal → Akhir)": "7 → 7",
      "Berat Akhir / Cap": "197 / 200",
      "Durasi (detik)": 1.911,
      "Total Iterasi": 200,
      "Jumlah Populasi": 20,
      "Banyak Iterasi": 200,
      "Rata-rata Fitness Akhir": 385.4,
      "Mutation Rate": 0.2
    },
    {
      "Run": "Pop=20, Iter=200 (Run #3)",
      "Objective Awal": 495,
      "Objective Akhir (Best)": 371,
      "Peningkatan (Delta)": -124,
      "Paket Masuk (Awal → Akhir)": "7 → 6",
      "Berat Akhir / Cap": "187 / 200",
      "Durasi (detik)": 1.8013,
      "Total Iterasi": 200,
      "Jumlah Populasi": 20,
      "Banyak Iterasi": 200,
      "Rata-rata Fitness Akhir": 319.7,
      "Mutation Rate": 0.2
    }
  ]
}
```


Iteration control:


```
{
  "algorithm": "Genetic Algorithm",
```

```
"total_runs": 9,
"experiment_logs": [
  {
    "Run": "Pop=10, Iter=100 (Run #1)",
    "Objective Awal": 510,
    "Objective Akhir (Best)": 313,
    "Peningkatan (Delta)": -197,
    "Paket Masuk (Awal → Akhir)": "8 → 6",
    "Berat Akhir / Cap": "147 / 200",
    "Durasi (detik)": 0.4933,
    "Total Iterasi": 100,
    "Jumlah Populasi": 10,
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 304.1,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=10, Iter=100 (Run #2)",
    "Objective Awal": 473,
    "Objective Akhir (Best)": 452,
    "Peningkatan (Delta)": -21,
    "Paket Masuk (Awal → Akhir)": "7 → 7",
    "Berat Akhir / Cap": "167 / 200",
    "Durasi (detik)": 0.5119,
    "Total Iterasi": 100,
    "Jumlah Populasi": 10,
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 295.3,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=10, Iter=100 (Run #3)",
    "Objective Awal": 495,
    "Objective Akhir (Best)": 505,
    "Peningkatan (Delta)": 10,
    "Paket Masuk (Awal → Akhir)": "7 → 7",
    "Berat Akhir / Cap": "174 / 200",
    "Durasi (detik)": 0.4781,
    "Total Iterasi": 100,
    "Jumlah Populasi": 10,
```

```
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 384.6,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=20, Iter=100 (Run #1)",
    "Objective Awal": 556,
    "Objective Akhir (Best)": 375,
    "Peningkatan (Delta)": -181,
    "Paket Masuk (Awal → Akhir)": "9 → 6",
    "Berat Akhir / Cap": "163 / 200",
    "Durasi (detik)": 0.8511,
    "Total Iterasi": 100,
    "Jumlah Populasi": 20,
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 328.45,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=20, Iter=100 (Run #2)",
    "Objective Awal": 473,
    "Objective Akhir (Best)": 413,
    "Peningkatan (Delta)": -60,
    "Paket Masuk (Awal → Akhir)": "7 → 7",
    "Berat Akhir / Cap": "197 / 200",
    "Durasi (detik)": 0.9052,
    "Total Iterasi": 100,
    "Jumlah Populasi": 20,
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 368.65,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=20, Iter=100 (Run #3)",
    "Objective Awal": 495,
    "Objective Akhir (Best)": 415,
    "Peningkatan (Delta)": -80,
    "Paket Masuk (Awal → Akhir)": "7 → 7",
    "Berat Akhir / Cap": "199 / 200",
    "Durasi (detik)": 1.2375,
```

```
    "Total Iterasi": 100,
    "Jumlah Populasi": 20,
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 367.05,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=30, Iter=100 (Run #1)",
    "Objective Awal": 568,
    "Objective Akhir (Best)": 374,
    "Peningkatan (Delta)": -194,
    "Paket Masuk (Awal → Akhir)": "8 → 6",
    "Berat Akhir / Cap": "172 / 200",
    "Durasi (detik)": 1.3545,
    "Total Iterasi": 100,
    "Jumlah Populasi": 30,
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 273.27,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=30, Iter=100 (Run #2)",
    "Objective Awal": 522,
    "Objective Akhir (Best)": 453,
    "Peningkatan (Delta)": -69,
    "Paket Masuk (Awal → Akhir)": "8 → 7",
    "Berat Akhir / Cap": "190 / 200",
    "Durasi (detik)": 1.4373,
    "Total Iterasi": 100,
    "Jumlah Populasi": 30,
    "Banyak Iterasi": 100,
    "Rata-rata Fitness Akhir": 361.2,
    "Mutation Rate": 0.2
  },
  {
    "Run": "Pop=30, Iter=100 (Run #3)",
    "Objective Awal": 544,
    "Objective Akhir (Best)": 444,
    "Peningkatan (Delta)": -100,
    "Paket Masuk (Awal → Akhir)": "9 → 7",
```

```
      "Berat Akhir / Cap": "141 / 200",
      "Durasi (detik)": 1.6404,
      "Total Iterasi": 100,
      "Jumlah Populasi": 30,
      "Banyak Iterasi": 100,
      "Rata-rata Fitness Akhir": 325.37,
      "Mutation Rate": 0.2
    }
  ]
}
```

Analisis

1. **Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?**
   Berdasarkan objective akhir pada eksperimen Hill Climbing, Hill Climbing with Sideways Move menghasilkan nilai tertinggi pada Test Case 1, yaitu 884, dengan 13 paket masuk dan berat 496/500. Pada Test Case 2, algoritma ini dan Random Restart Hill Climbing sama-sama mencapai 549. Pada Test Case 3, keempat variasi Hill Climbing yang dicatat mencapai nilai yang sama, yaitu 587.

   Perbedaan ini dapat dijelaskan melalui mekanisme pencarian. Steepest Ascent Hill Climbing selalu memilih tetangga dengan objective terbaik, tetapi berhenti ketika tidak ada tetangga yang lebih baik sehingga rentan terjebak pada local optimum. Stochastic Hill Climbing memeriksa tetangga secara acak dan menerima tetangga pertama yang lebih baik; pendekatan ini lebih murah per langkah, tetapi tidak selalu menemukan perbaikan terbaik. Sideways Move memungkinkan perpindahan ke state dengan nilai sama sehingga pencarian dapat melewati plateau. Random Restart mengulang pencarian dari state awal baru ketika mencapai local optimum dan menyimpan solusi terbaik yang pernah ditemukan, sehingga memperoleh lebih banyak peluang menemukan solusi yang lebih baik.

2. **Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?**
   Pada Test Case 1, Sideways Move memperoleh objective akhir tertinggi (884), diikuti Random Restart (811), Steepest Ascent (738), dan Stochastic Hill Climbing (596). Pada Test Case 2, Sideways Move dan Random Restart sama-sama memperoleh 549, lebih tinggi daripada Stochastic (529) dan Steepest Ascent (426). Pada Test Case 3, semua variasi Hill Climbing mencapai 587. Pola ini menunjukkan bahwa tidak ada satu variasi yang unggul pada setiap kasus, walaupun Sideways Move dan Random Restart memberi hasil lebih tinggi pada dua kasus pertama.

   Genetic Algorithm menggunakan populasi solusi dan operator selection, crossover, serta mutation, sehingga pendekatannya berbeda dari local search yang memperbaiki satu state atau lintasan pencarian. Pada eksperimen Test

Case 3 dengan populasi 20 dan mutation rate 0,2, objective akhir yang dilaporkan berkisar 324–465 untuk pengendalian populasi dan 313–505 untuk pengendalian iterasi. Nilai tersebut tidak dapat dibandingkan langsung dengan nilai 587 pada Test Case 3 Hill Climbing tanpa memastikan bahwa konfigurasi input, objective awal, dan kondisi eksperimennya identik.

3. **Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?**

   Stochastic Hill Climbing cepat karena mengambil tetangga secara acak, bukan mengevaluasi seluruh tetangga untuk menentukan yang terbaik. Steepest Ascent harus mengevaluasi semua tetangga pada setiap langkah. Sideways Move dapat terus menjelajahi state dengan nilai sama hingga batas gerakan sideways tercapai, sedangkan Random Restart menjalankan pencarian berulang dari beberapa state awal. Kedua mekanisme tersebut menambah biaya komputasi, tetapi dapat menghasilkan solusi lebih baik pada sebagian kasus. Durasi Genetic Algorithm meningkat seiring jumlah generasi atau ukuran populasi yang diuji: misalnya pada pengendalian iterasi, durasi sekitar 0,49–0,51 detik untuk populasi 10, 0,85–1,24 detik untuk populasi 20, dan 1,35–1,64 detik untuk populasi 30 (semuanya 100 iterasi). Hasil ini konsisten dengan bertambahnya jumlah individu yang perlu dievaluasi dan diproses. Durasi Simulated Annealing tidak tercantum, sehingga tidak dapat dibandingkan secara numerik.

4. **Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?**
   Untuk algoritma seperti Steepest Ascent Hill Climbing hasil selalu deterministik. Hal ini karena untuk setiap *state* selalu dibangkitkan seluruh *neighbornya* dan dipastikan dipilih *neighbor* paling bagus. Beberapa algoritma Hill Climb lain juga mengikuti ini kecuali *stochastic* dan *random restart*. Selain hill climb seperti Simulated Annealing dan Genetic Algorithm dipastikan selalu ada elemen acak pada hasil akhir sehingga hasil akhir tidak selalu deterministik.

   Eksperimen Genetic Algorithm memperlihatkan variasi hasil yang nyata. Pada pengendalian populasi (populasi 20, mutation rate 0,2), objective akhir untuk 50 iterasi adalah 324, 411, dan 449; untuk 100 iterasi adalah 412, 424, dan 410;

sedangkan untuk 200 iterasi adalah 408, 465, dan 371. Rentang hasil ini menunjukkan bahwa run dengan konfigurasi sama dapat menghasilkan solusi berbeda, kemungkinan karena proses inisialisasi dan operator genetik melibatkan unsur acak. Pada pengendalian iterasi dengan 100 iterasi, populasi 10 menghasilkan 313, 452, dan 505; populasi 20 menghasilkan 375, 413, dan 415; populasi 30 menghasilkan 374, 453, dan 444. Hasil tersebut juga menunjukkan variasi antar-run.

5. **Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?**
    - Pengaruh jumlah iterasi

Pada pengujian dengan populasi 20 dan mutation rate 0,2, peningkatan jumlah iterasi dari 50 menjadi 100 lalu 200 tidak menghasilkan kenaikan objective akhir secara konsisten. Rata-rata objective akhir dari tiga run adalah sekitar 394,7 untuk 50 iterasi, 415,3 untuk 100 iterasi, dan 414,7 untuk 200 iterasi. Dengan demikian, 100 iterasi menghasilkan rata-rata tertinggi pada data ini, tetapi selisihnya kecil terhadap 200 iterasi dan hasil tiap run bervariasi. Menambah iterasi juga meningkatkan durasi: sekitar 0,47–0,68 detik pada 50 iterasi, 1,06–1,20 detik pada 100 iterasi, dan 1,80–2,05 detik pada 200 iterasi. Jadi, pada eksperimen ini, iterasi tambahan meningkatkan biaya komputasi tetapi tidak menjamin solusi akhir yang lebih baik.

- Pengaruh jumlah populasi

Pada pengujian 100 iterasi dengan mutation rate 0,2, rata-rata objective akhir dari tiga run adalah sekitar 423,3 untuk populasi 10, 401,0 untuk populasi 20, dan 423,7 untuk populasi 30. Rata-rata tertinggi pada data ini diperoleh populasi 30, tetapi nilainya hampir sama dengan populasi 10 dan tidak menunjukkan tren peningkatan yang konsisten terhadap ukuran populasi. Populasi lebih besar menyediakan lebih banyak kandidat solusi per generasi, tetapi juga menambah biaya evaluasi. Pada hasil yang dilaporkan, durasi meningkat dari sekitar 0,48–0,51 detik untuk populasi 10 menjadi 0,85–1,24 detik untuk populasi 20 dan 1,35–1,64 detik untuk populasi 30.

Kesimpulannya, pada konfigurasi yang diuji, baik penambahan iterasi maupun populasi tidak menjamin peningkatan objective akhir. Karena hanya ada tiga run per konfigurasi, kesimpulan ini bersifat indikatif. Evaluasi yang lebih kuat perlu menggunakan lebih banyak pengulangan dengan seed dan kondisi awal

yang tercatat, serta melaporkan best-so-far, rata-rata, simpangan baku, dan durasi.

Kesimpulan dan Saran

Sebagian variasi dari Hill Climb, Simulated Annealing, dan Genetic Algorithm mampu mendekati objective function layaknya algoritma pada umumnya. Hal ini karena permasalahan *truck loading* ini cocok untuk ketiga algoritma ini. Permasalahan *truck loading* ini memiliki banyak elemen *trial and error* yang mana merupakan syarat bisa dilakukannya algoritma Hill Climb, Simulated Annealing, dan Genetic Algorithm.

Setelah menyelesaikan tugas ini, saran yang bisa diberikan adalah dengan menspesifikasikan beberapa *term* yang kurang jelas. Contohnya seperti frekuensi *stuck* pada SImulated Annealing yang tidak didefinisikan dengan jelas baik di spesifikasi tugas besar, maupun di kelas.

Pembagian Tugas

NIM	Tugas
13524024	Simulated Annealing + Visualisasi + Dashboard
13524048	Semua Hill Climbing + Testing + Laporan
13524088	Genetic Algorithm + Laporan

Referensi

https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/
https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/
https://www.geeksforgeeks.org/dsa/genetic-algorithms/
https://www.geeksforgeeks.org/machine-learning/crossover-in-genetic-algorithm/

https://plotly.com/python-api-reference/
https://docs.streamlit.io/develop/api-reference

Lampiran

Pernyataan penggunaan AI:

Pernyataan AI Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027